

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

In this experiment, I studied how CNN architectures have developed over time and compared their behaviour on the CIFAR-10 image classification task. I worked with LeNet-5, AlexNet, VGG16, GoogLeNet and ResNet, and used transfer learning with the pretrained models. I also compared their classification results, parameter counts and training times.

2. Learning Outcomes

By doing this experiment, I was able to understand the main ideas behind the different CNN architectures, distinguish between the models, use pretrained networks for transfer learning, fine tune them on CIFAR-10, and compare their performance using standard metrics.

3. Introduction

CNNs are widely used for image classification because they can learn useful features such as edges, textures and shapes directly from images. As CNNs developed, different architectures were introduced to improve accuracy and to make deeper networks easier or more efficient to train.

In this experiment, I followed the progression from LeNet-5 to AlexNet, VGG16, GoogLeNet and ResNet. Each model uses a different idea to improve convolutional networks.

I implemented the experiment in **PyTorch** and used `torchvision.models` for the pretrained architectures and their ImageNet weights.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogLeNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 is one of the early CNN architectures and was originally designed for handwritten digit recognition. It uses convolution layers, average pooling and fully connected layers to learn image features. In this experiment, I used a LeNet-5 style network as the from-scratch baseline.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

I trained LeNet-5 from scratch without pretrained weights. I used it as the baseline so that I could compare a small CNN trained only on CIFAR-10 with the larger pretrained models.

6. AlexNet

AlexNet became an important milestone in CNN development after its strong performance in the 2012 ImageNet competition. It popularized ReLU activations, Dropout and GPU-based training. I included it to compare an older deep CNN with the newer architectures.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 uses a simple pattern of stacking small 3×3 convolution filters. Two such layers can cover a receptive field similar to a larger filter while also adding another nonlinear operation. I included VGG16 to see how this simple but deep design performs in the comparison.

Advantages

- Excellent feature extraction
- Simple, uniform architecture
- High classification accuracy

8. Comparison of Architectures (Published Reference Values)

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

Note: the parameter counts above are the standard published ImageNet figures (1000-class output). The models used in this experiment have their final classifier replaced by a 10-way layer, so their actual parameter counts differ slightly. ResNet18 (not ResNet50) was used here for tractable training time on a single T4 GPU.

9. GoogLeNet (Inception Network)

GoogLeNet introduced the Inception module, where different convolution operations are performed in parallel. It uses different filter sizes so that the network can capture features at different scales. The 1×1 convolutions also reduce the amount of computation before the larger filters.

GoogLeNet also uses two **auxiliary classifiers** attached to intermediate layers during training. Their losses are added to the main loss with a weight of 0.3:

$$L_{total} = L_{main} + 0.3 L_{aux1} + 0.3 L_{aux2}$$

The auxiliary classifiers provide extra learning signals during training and are not needed when making the final predictions. During implementation, I handled GoogLeNet separately because the PyTorch model returns the main output together with the auxiliary outputs during training.

Advantages

- Multi-scale feature extraction.
- Far fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

As CNNs become deeper, training can become difficult because gradients may become very small. ResNet addresses this using **skip (residual) connections**, which give information and gradients a shorter path through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

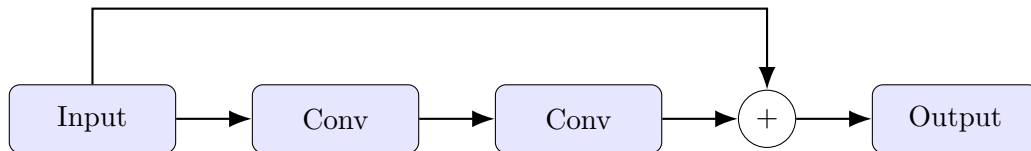


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution increases the receptive field without increasing the number of kernel parameters. I studied it as another way of changing what part of an image a convolution can see.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution is used to increase the spatial dimensions of feature maps. Unlike pooling, which reduces image size, it performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

For transfer learning, I started with CNN models that had already learned general image features from ImageNet. This gave the pretrained models a useful starting point instead of making them learn all the visual features from random weights.

The procedure I followed was:

1. Load a pretrained model.
2. Remove the original classifier.
3. Attach a new classification head matching the target number of classes.
4. Either freeze the convolutional base (feature extraction) or train all layers at a low learning rate (full fine tuning).
5. Train and evaluate.

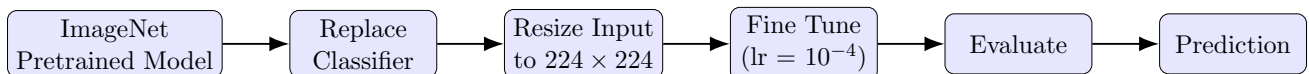


Figure 2: Transfer Learning Workflow Used in This Experiment

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

13.1 Strategy Adopted in This Experiment

Two transfer-learning strategies are possible:

	Feature Extraction	Full Fine Tuning
Convolutional base	Frozen	Trainable
Trainable parameters	Only the new head	Entire network
Learning rate	Normal (10^{-3})	Low (10^{-4})
Cost per epoch	Low	High
Accuracy on CIFAR-10	Lower	Higher

I used **full fine tuning** for the pretrained models. I replaced the final classification layer with a 10-class `nn.Linear` layer and trained the complete network using Adam with a learning rate of 10^{-4} . I used the smaller learning rate so that the useful ImageNet features would not change too aggressively.

For LeNet-5, which started from random weights, I used a learning rate of 10^{-3} .

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

14.1 Two Preprocessing Pipelines

The original CIFAR-10 images are 32×32 pixels, while the pretrained models expect 224×224 inputs and ImageNet normalization. Because of this difference, I used two separate preprocessing pipelines:

Pipeline	Used by	Operations
<code>transform_32</code>	LeNet-5	<code>ToTensor()</code> ; Normalize with CIFAR-10 statistics mean (0.4914, 0.4822, 0.4465), std (0.2470, 0.2435, 0.2616)
<code>transform_224</code>	AlexNet, VGG16, GoogLeNet, ResNet18	<code>Resize((224,224))</code> ; <code>ToTensor()</code> ; Normalize with ImageNet statistics mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225)

Why I used these settings. The pretrained networks were originally trained with ImageNet normalization, so I kept the same normalization when using their weights. I also resized the CIFAR-10 images to 224×224 so that the pretrained architectures could process them properly.

Hardware: NVIDIA Tesla T4 GPU, PyTorch 2.11.0+cu128, Torchvision 0.26.0+cu128.

GitHub Repository: <https://github.com/ReshmanthCH/deep-learning>

15. Experimental Procedure

Task 1: Dataset Preparation

I first loaded the CIFAR-10 dataset using `torchvision.datasets`. I used the appropriate channel statistics to normalize the images, displayed ten sample images to check the data, and printed the sizes of the training and testing sets.

Output.

PyTorch version: 2.11.0+cu128
Torchvision version: 0.26.0+cu128
Device: cuda
GPU: Tesla T4

Training images: 50000
Testing images: 10000



Figure 3: Sample CIFAR-10 Images with Class Labels (Mandatory Plot 1)

What I observed: The dataset contains low-resolution 32×32 RGB images spanning all ten classes, and the printed counts confirm 50,000 training and 10,000 testing images. Objects remain recognizable but fine detail is already lost at this resolution, which is exactly why the small-object classes (cat, dog, bird) turn out to be the hardest ones later.

Task 2: Baseline CNN from Scratch (LeNet-5)

I implemented the LeNet-5 baseline with two convolution and average-pooling stages followed by three fully connected layers. I trained it on the 32×32 images for five epochs using Adam with a learning rate of 10^{-3} and a batch size of 128.

Output: Model Definition.

```
LeNet5(  
    (features): Sequential(  
      (0): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))  
      (1): ReLU()  
      (2): AvgPool2d(kernel_size=2, stride=2, padding=0)  
      (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))  
      (4): ReLU()  
      (5): AvgPool2d(kernel_size=2, stride=2, padding=0)  
    )  
    (classifier): Sequential(  
      (0): Flatten(start_dim=1, end_dim=-1)  
      (1): Linear(in_features=400, out_features=120, bias=True)  
      (2): ReLU()  
      (3): Linear(in_features=120, out_features=84, bias=True)  
      (4): ReLU()  
    )  
)
```

```

    (5): Linear(in_features=84, out_features=10, bias=True)
  )
)

```

Output: Training Log.

```

Epoch [1/5] Loss: 1.7303 Accuracy: 36.73% Time: 13.2s
Epoch [2/5] Loss: 1.4251 Accuracy: 48.69% Time: 12.1s
Epoch [3/5] Loss: 1.3261 Accuracy: 52.40% Time: 11.9s
Epoch [4/5] Loss: 1.2604 Accuracy: 54.81% Time: 11.6s
Epoch [5/5] Loss: 1.2056 Accuracy: 56.98% Time: 11.9s

```

What I observed: LeNet-5 reaches only 56.98% training accuracy after 5 epochs and its loss is still falling slowly. With roughly 62K parameters and only two convolution layers it simply does not have the capacity to model CIFAR-10's colour, texture and pose variation. It is, however, extremely cheap – about 12 seconds per epoch, over 60 times faster per epoch than VGG16 – which establishes the accuracy/cost trade-off that the rest of the experiment explores.

Task 3: Transfer Learning with Pretrained Architectures

For each pretrained model, I followed the same basic procedure. I loaded the ImageNet weights using `torchvision.models`, replaced the final classifier with a 10-class `nn.Linear` layer, used the 224×224 ImageNet-normalized input pipeline with a batch size of 64, and trained the model using Adam with a learning rate of 10^{-4} and cross-entropy loss.

Head replacement performed per model:

Model	Layer replaced
AlexNet	<code>classifier[6]</code> : 4096 $\rightarrow$ 10
VGG16	<code>classifier[6]</code> : 4096 $\rightarrow$ 10
GoogLeNet	<code>fc</code> , <code>aux1.fc2</code> , <code>aux2.fc2</code> : $\rightarrow$ 10
ResNet18	<code>fc</code> : 512 $\rightarrow$ 10

Output: AlexNet Training Log (5 epochs, lr = 0.0001).

```

Epoch [1/5] Loss: 0.5249 Accuracy: 81.65% Time: 91.1s
Epoch [2/5] Loss: 0.2832 Accuracy: 90.20% Time: 90.8s
Epoch [3/5] Loss: 0.1889 Accuracy: 93.44% Time: 93.0s
Epoch [4/5] Loss: 0.1358 Accuracy: 95.16% Time: 92.1s
Epoch [5/5] Loss: 0.1019 Accuracy: 96.45% Time: 93.1s

```

What I observed: AlexNet starts at 81.65% after a *single* epoch – higher than LeNet-5 ever reaches in five. This gap is the whole point of transfer learning: the edge, texture and shape detectors learned on ImageNet transfer directly to CIFAR-10, so the network only has to learn the mapping from those features to ten new labels.

Output: VGG16 Training Log (3 epochs, lr = 0.0001).

```

Epoch [1/3] Loss: 0.4340 Accuracy: 85.17% Time: 743.7s
Epoch [2/3] Loss: 0.1969 Accuracy: 93.31% Time: 740.6s
Epoch [3/3] Loss: 0.1215 Accuracy: 95.79% Time: 743.3s

```

What I observed: VGG16 converges quickly in terms of epochs but costs about 744s per epoch – roughly eight times AlexNet – because its 138M parameters and uniformly deep 3×3 stacks make every forward and backward pass expensive. It was therefore trained for 3 epochs rather than 5.

Output: GoogLeNet Training Log (3 epochs, lr = 0.0001, auxiliary loss).

```
Epoch [1/3] Loss: 0.9144 Accuracy: 88.10% Time: 192.3s
Epoch [2/3] Loss: 0.3749 Accuracy: 96.38% Time: 190.5s
Epoch [3/3] Loss: 0.2410 Accuracy: 98.34% Time: 190.8s
```

What I observed: GoogLeNet reaches the highest training accuracy of the pretrained group (98.34%) at about a quarter of VGG16's cost per epoch. Its reported loss values are *not* directly comparable with the other models, because the logged loss is the combined $L_{main} + 0.3L_{aux1} + 0.3L_{aux2}$ rather than a plain cross-entropy.

Output: ResNet18 Training Log (3 epochs, lr = 0.0001).

```
Epoch [1/3] Loss: 0.3188 Accuracy: 89.46% Time: 150.2s
Epoch [2/3] Loss: 0.0951 Accuracy: 96.96% Time: 149.2s
Epoch [3/3] Loss: 0.0417 Accuracy: 98.74% Time: 149.4s
```

What I observed: ResNet18 gives the best training accuracy (98.74%) and the lowest training loss (0.0417) of any model, at only ≈ 150 s per epoch. Residual connections let the optimizer make fast progress even in a deep network, and with only 11M parameters ResNet18 is far cheaper than VGG16 while fitting the data better.

Task 4: Model Evaluation

I evaluated each fine-tuned model on the 10,000-image test set using

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1-score (weighted)
- Confusion Matrix
- Classification Report

Output: Per-Model Test Metrics.

AlexNet

```
Accuracy : 0.9131
Precision: 0.9139792486302826
Recall    : 0.9131
F1 Score  : 0.9125217857650827
```

VGG16

```
Accuracy : 0.9251
Precision: 0.9265129465733396
Recall    : 0.9251
F1 Score  : 0.9250478815368323
```

GoogLeNet

```
Accuracy : 0.9508
Precision: 0.9508634240083935
Recall    : 0.9508
F1 Score  : 0.9507334345974897
```

ResNet18

```
Accuracy : 0.9406
Precision: 0.9423406819012337
Recall    : 0.9406
F1 Score  : 0.9406708934391154
```

What I observed: For every model, precision, recall and F1 sit within ± 0.002 of accuracy. Since CIFAR-10's test set is perfectly balanced (1000 images per class), this agreement confirms that no model is achieving its score by favouring a few easy classes.

Output: Consolidated Results Table.

	Accuracy	Precision	Recall	F1-Score
AlexNet	0.9131	0.913979	0.9131	0.912522
VGG16	0.9251	0.926513	0.9251	0.925048
GoogLeNet	0.9508	0.950863	0.9508	0.950733
ResNet18	0.9406	0.942341	0.9406	0.940671

Output: Classification Report (ResNet18).

	precision	recall	f1-score	support
airplane	0.93	0.97	0.95	1000
automobile	0.98	0.96	0.97	1000
bird	0.95	0.92	0.93	1000
cat	0.92	0.82	0.87	1000
deer	0.94	0.95	0.95	1000
dog	0.83	0.94	0.88	1000
frog	0.98	0.95	0.97	1000
horse	0.94	0.97	0.95	1000
ship	0.97	0.95	0.96	1000
truck	0.97	0.96	0.97	1000
accuracy			0.94	10000
macro avg	0.94	0.94	0.94	10000
weighted avg	0.94	0.94	0.94	10000

What I observed: The report isolates a single weak pair. **cat** has high precision (0.92) but low recall (0.82), while **dog** has the opposite profile – low precision (0.83) but high recall (0.94). That asymmetry means the model is biased towards predicting “dog” when it is unsure, so a large number of true cats are absorbed into the dog class. Every other class scores 0.93 or above.

16. Hyperparameter Settings

The settings I used for each model are listed below.

Model	Weights	Input	Batch	Epochs	Learning Rate
LeNet-5	Random init	32×32	128	5	0.001
AlexNet	ImageNet	224×224	64	5	0.0001
VGG16	ImageNet	224×224	64	3	0.0001
GoogLeNet	ImageNet	224×224	64	3	0.0001
ResNet18	ImageNet	224×224	64	3	0.0001

Common to all runs:

- Optimizer: Adam
- Loss: Cross-Entropy (`nn.CrossEntropyLoss`, which applies log-softmax internally, so the models output raw logits)
- Metric: Accuracy, plus weighted Precision / Recall / F1
- `num_workers = 2`, `pin_memory = True`

Why I used two learning rates. I used 10^{-4} for the pretrained models and 10^{-3} for LeNet-5. The pretrained models already had useful weights, so I used the smaller learning rate during fine tuning. LeNet-5 started from random weights, so the larger learning rate was reasonable for the baseline.

Why I used different epoch counts. I trained VGG16, GoogLeNet and ResNet18 for three epochs because VGG16 was expensive to run. Three epochs of VGG16 took about 37 minutes, and the pretrained models had already crossed 95% training accuracy by the third epoch. LeNet-5 and AlexNet were faster, so I trained them for five epochs.

17. Mandatory Plots

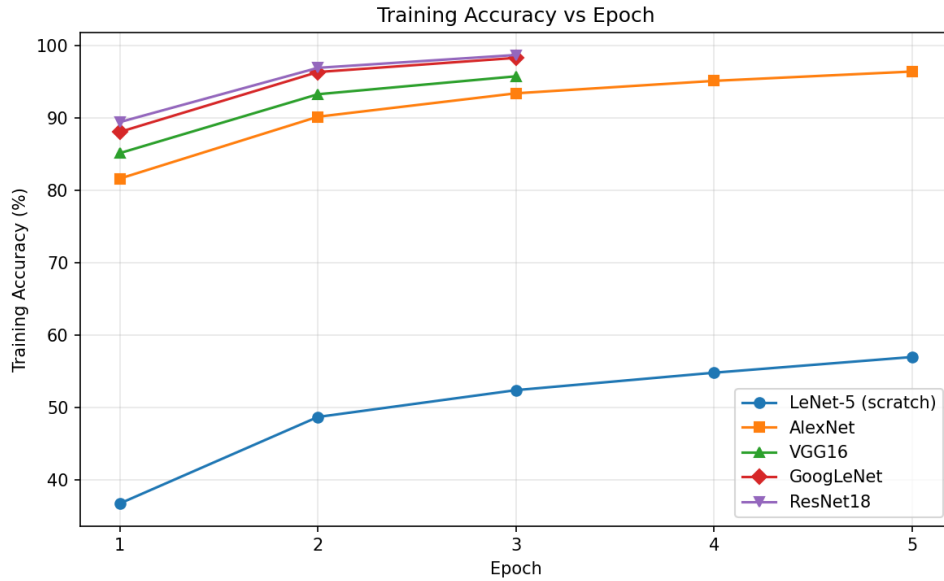


Figure 4: Training Accuracy vs Epoch for All Five Architectures (Mandatory Plot 2)

The training-accuracy plot shows a clear difference between the from-scratch model and the pretrained models. LeNet-5 stayed in the 37–57% range, while the pretrained models started above 81% and improved quickly. ResNet18 and GoogLeNet finished with the highest training accuracies.

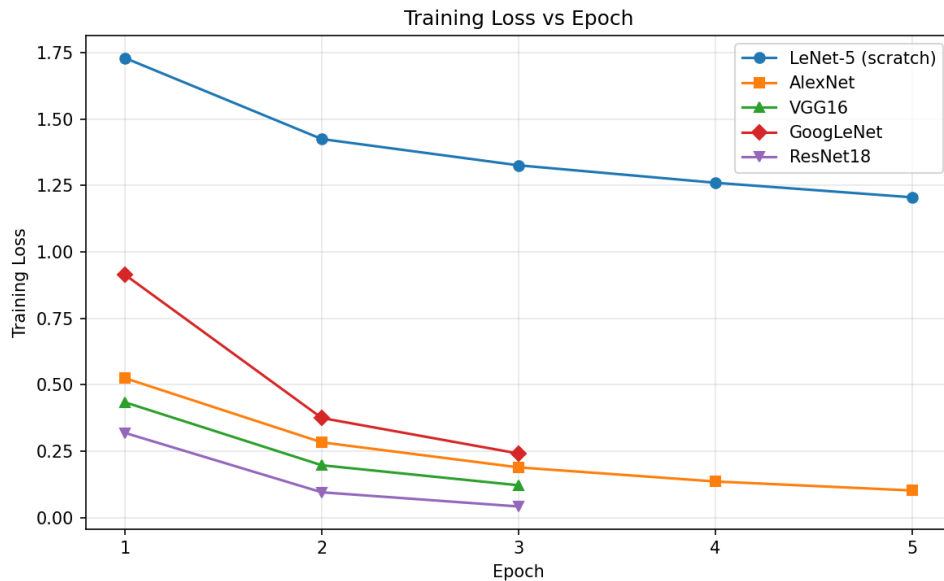


Figure 5: Training Loss vs Epoch for All Five Architectures (Mandatory Plot 3)

The loss decreased quickly for the pretrained models, especially ResNet18, whose loss reached 0.0417 by the third epoch. LeNet-5 still had a loss above 1.2 after five epochs. GoogLeNet is different because its training loss also includes the two auxiliary classifier losses.

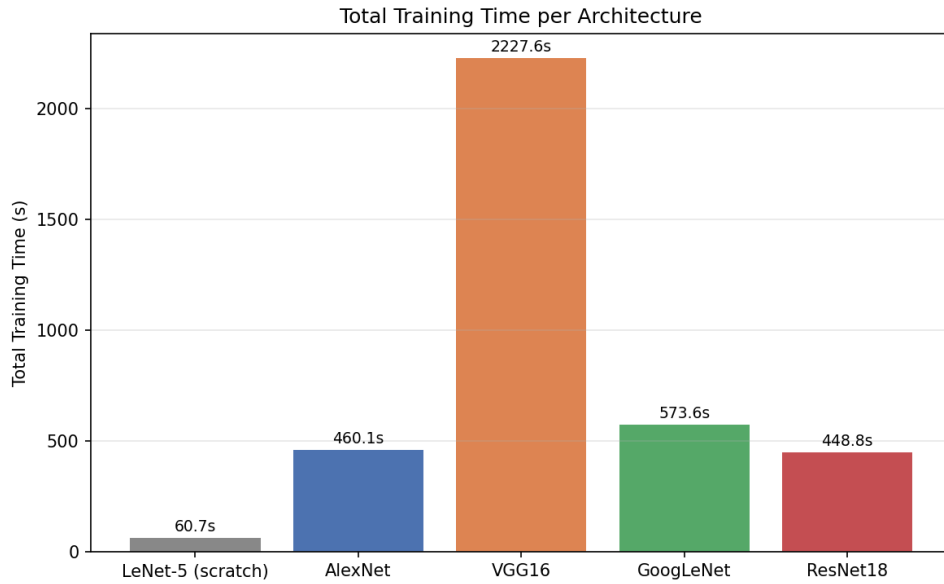


Figure 6: Total Training Time per Architecture (Mandatory Plot 4)

The training-time plot shows that the model taking the longest to train was not necessarily the one with the best test result. VGG16 took 2227.6s in total, almost five times the time taken by ResNet18. Even with this extra training time, VGG16 did not achieve the best test accuracy.

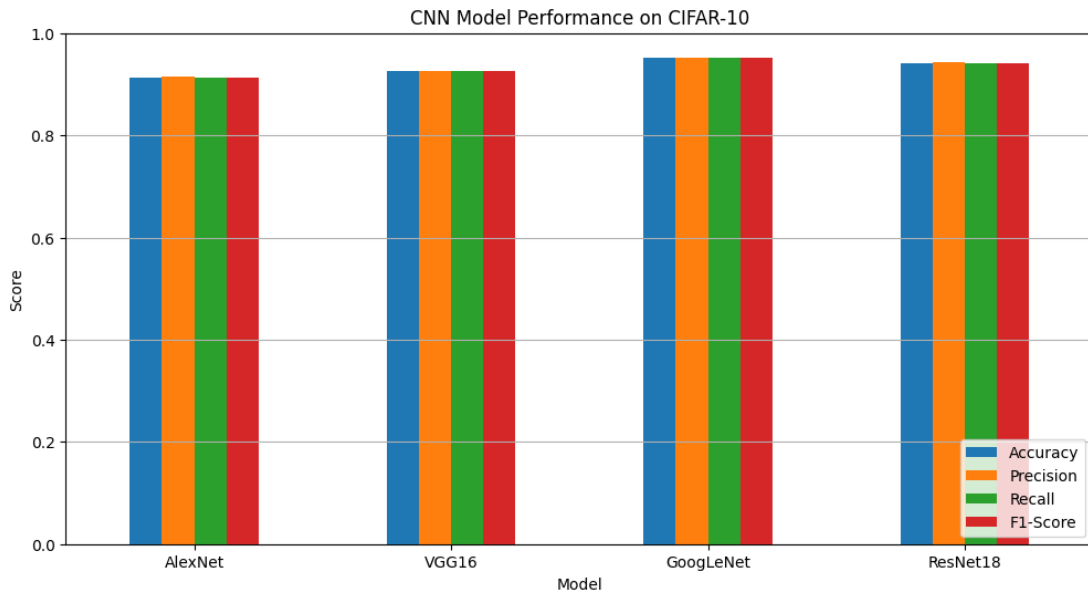


Figure 7: Test Accuracy, Precision, Recall and F1-Score by Model (Mandatory Plot 5)

The four evaluation metrics are very close to one another for each model. Among the pretrained models, GoogLeNet gave the highest test accuracy, followed by ResNet18, VGG16 and AlexNet. The difference between the highest and lowest pretrained-model accuracies was about 3.8 percentage points.

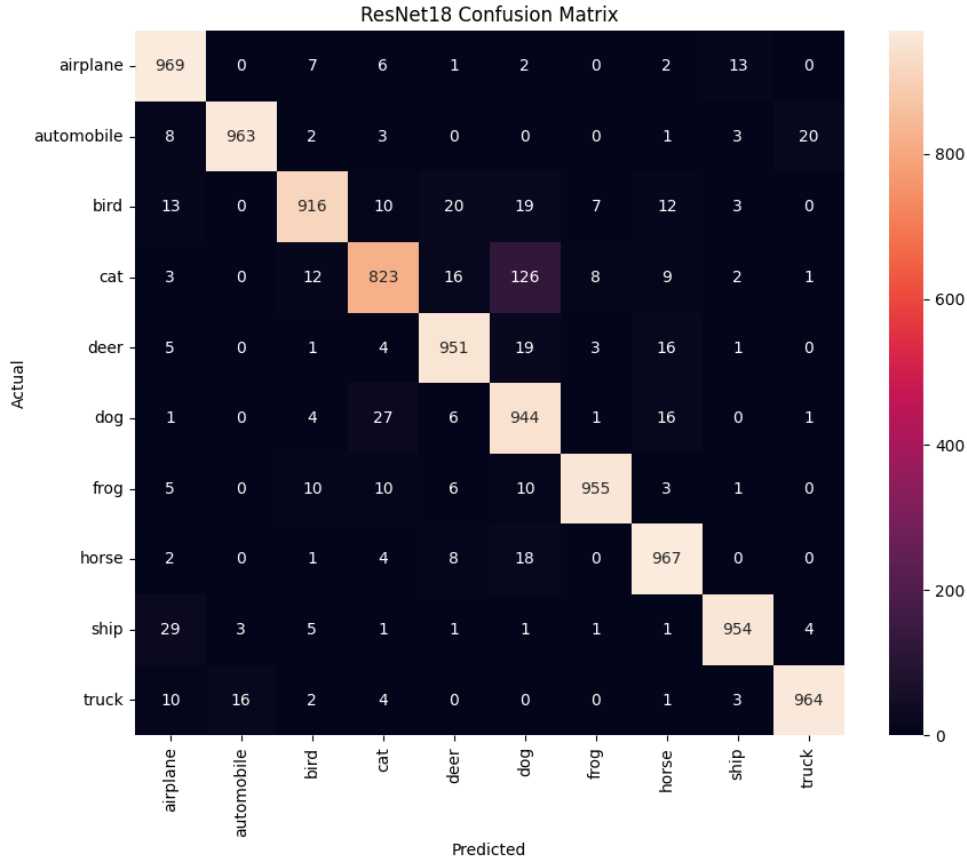


Figure 8: ResNet18 Confusion Matrix on the CIFAR-10 Test Set (Mandatory Plot 6)

The confusion matrix is mostly diagonal, so most test images were classified correctly. The largest confusion was between cats and dogs: 126 cat images were predicted as dogs, while 27 dog images were predicted as cats. There were also smaller confusions between visually related classes such as ship and airplane, automobile and truck, and bird and deer.

Note on validation curves. I did not keep a separate validation split in this run, so validation accuracy and validation loss were not recorded for each epoch. Instead, I used the 10,000-image test set to check how well the trained models generalized. A validation split and early stopping could be added in a future run.

18. Results

18.1 Final Comparison

Model	Test Acc.	Precision	Recall	F1-Score	Epochs	Train Time (s)
LeNet-5 (scratch)	Not evaluated	–	–	–	5	60.7
AlexNet	0.9131	0.9140	0.9131	0.9125	5	460.1
VGG16	0.9251	0.9265	0.9251	0.9250	3	2227.6
GoogLeNet	0.9508	0.9509	0.9508	0.9507	3	573.6
ResNet18	0.9406	0.9423	0.9406	0.9407	3	448.8

I used LeNet-5 mainly as the from-scratch baseline, so I did not calculate its test-set metrics in this run. Its final training accuracy was 56.98%, which was much lower than the pretrained models.

18.2 Accuracy per Second of Training

Model	Test Accuracy	Train Time (s)	Accuracy per 100 s
AlexNet	0.9131	460.1	0.198
VGG16	0.9251	2227.6	0.042
GoogLeNet	0.9508	573.6	0.166
ResNet18	0.9406	448.8	0.210

When training time is taken into account, ResNet18 gave the best balance in this experiment. GoogLeNet had the highest test accuracy, but it required more training time than ResNet18. VGG16 was the least efficient because it took much longer without giving the highest test accuracy.

18.3 Best Model

Best test accuracy	GoogLeNet – 95.08%
Best accuracy/cost ratio	ResNet18 – 94.06% in 448.8 s
Weakest pretrained model	AlexNet – 91.31%
Baseline (from scratch)	LeNet-5 – 56.98% training accuracy

19. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?** AlexNet was a major milestone because it showed that a deeper CNN could achieve a large improvement in image classification when trained with ReLU activations, Dropout and GPUs. These ideas made it practical to train much larger networks. In my experiment, AlexNet also benefited from its ImageNet-pretrained weights and reached 91.31% test accuracy.
2. **Why does VGG16 use only 3×3 convolution filters?** VGG16 uses small 3×3 filters repeatedly instead of relying on larger filters. Two 3×3 convolutions can cover a receptive field similar to a 5×5 filter while adding another nonlinear operation. This keeps the convolution blocks simple, although VGG16 still has a large number of parameters.
3. **Explain the advantages of the Inception module.** The Inception module runs different filter sizes in parallel and combines their outputs. This allows the network to capture features at different scales in the same layer. The 1×1 convolutions also reduce the number of channels before the larger convolutions. In my results, GoogLeNet reached the highest test accuracy of 95.08%.
4. **What is the purpose of residual learning?** Residual learning makes a layer learn the difference between the desired output and the original input. The output is then formed as $F(x) + x$. The skip connection also gives the gradient a direct path through the network, which makes deeper models easier to train.
5. **Differentiate LeNet and ResNet.** LeNet-5 is a small and relatively simple CNN, while ResNet18 is much deeper and uses residual connections and batch normalization. LeNet-5 was originally designed for small grayscale digit images, whereas ResNet was designed for more complex natural images. The difference was also visible in my experiment: LeNet-5 reached 56.98% training accuracy, while ResNet18 reached 94.06% test accuracy.
6. **What is Transfer Learning?** Transfer learning means starting with a model that has already learned useful features from a large dataset and adapting it to a new task. The early layers usually contain general features such as edges and textures, so they do not have to be learned again from random initialization. This helped the pretrained models start with much higher accuracy than the from-scratch baseline.
7. **Why is fine tuning required?** The features learned from ImageNet are useful, but they are not a perfect match for CIFAR-10. Fine tuning lets the network adjust its later features to the new classes while keeping useful information that was already learned. I used a small learning rate so that these changes happened gradually.

8. **Explain the difference between dilated convolution and transpose convolution.** Dilated convolution increases the receptive field by spacing out the kernel elements while keeping the feature-map resolution. Transpose convolution is used to increase the spatial resolution through learnable upsampling. In simple terms, dilation helps the network see a larger area, while transpose convolution helps produce a larger feature map.
9. **Why do pretrained models converge faster?** The pretrained models started with weights that had already learned useful visual features from ImageNet. Because of this, training did not have to begin from completely random features. The difference was clear in my logs: AlexNet reached 81.65% training accuracy after one epoch, while LeNet-5 was only at 36.73% after its first epoch.
10. **Compare the computational complexity of LeNet and ResNet.** LeNet-5 has only about 62K parameters and uses 32×32 inputs, so it was very quick to train. ResNet18 has about 11M parameters and uses 224×224 inputs, so its training time was much higher. In my run, LeNet-5 took about 12 seconds per epoch, while ResNet18 took about 149 seconds per epoch.

20. Additional Exercises and Possible Extensions

1. Hold out a validation split from the 50,000 training images to produce per-epoch validation accuracy and loss curves, and to enable early stopping.
2. Evaluate LeNet-5 on the test set so that the from-scratch baseline appears in the metric comparison, not only in the training log.
3. Run the frozen-base (feature-extraction) variant of each pretrained model and compare it against the full fine-tuning results reported here.
4. Compare Adam and SGD with momentum at the same learning rate.
5. Add data augmentation (`RandomCrop`, `RandomHorizontalFlip`) to reduce the cat/dog confusion.
6. Plot the misclassified images to inspect the cat/dog failure cases qualitatively.
7. Repeat with ResNet50 and EfficientNetB0 to extend the depth axis of the comparison.

21. Source Code

The complete source code and Google Colab implementation for this experiment are maintained in the following GitHub repository:

<https://github.com/ReshmanthCH/deep-learning>

The repository contains the files I used for the CNN architecture comparison, transfer learning, fine tuning and model evaluation.

22. Conclusion

In this experiment, I compared CNN architectures from different stages of CNN development using the CIFAR-10 dataset. I trained LeNet-5 from scratch and used ImageNet-pretrained AlexNet, VGG16, GoogLeNet and ResNet18 for transfer learning and fine tuning.

The results showed a clear advantage for the pretrained models. LeNet-5 reached 56.98% training accuracy after five epochs, while the pretrained models reached much higher training and test performance in fewer epochs. GoogLeNet gave the highest test accuracy at 95.08%, while ResNet18 gave a good balance between accuracy and training time, reaching 94.06% in 448.8 seconds.

VGG16 performed well but required considerably more training time. The ResNet18 confusion matrix also showed that the main remaining problem was distinguishing visually similar classes, especially cats and dogs. Overall, this experiment helped me understand how transfer learning can improve training on a new dataset and how architectural choices affect both accuracy and computational cost.

Configuration I would choose: For a practical setup, I would use ResNet18 with ImageNet weights, full fine tuning at a learning rate of 10^{-4} , Adam, batch size 64, and 224×224 ImageNet-normalized

inputs. If the main priority were the highest test accuracy rather than training time, I would choose GoogLeNet based on the results of this run.

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Alex Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, Technical Report, University of Toronto, 2009 (CIFAR-10).
7. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
8. PyTorch Documentation: <https://pytorch.org/docs>
9. Torchvision Models Documentation: <https://pytorch.org/vision/stable/models.html>